

Mutual Hazard Networks con EvAM-Tools y Python - Grupo 6

Tania Gonzalo Santana
Claudia Guerrero Rodríguez
Sandra Mingo Ramírez
Daniel Parra Gutiérrez
2024/25

Índice general

1	Introducción a Mutual Hazard Networks	2
2	Comparación de implementaciones entre R y Python	3
2.1	Funcionalidad	3
2.2	Velocidad	6
2.3	Ventajas y desventajas	6
3	Exploración de posibles integraciones	6
3.1	Integración del código en Python en R (Evamtools)	6
3.2	Modificar evamtools para incluir la corrección del sesgo	11

1. Introducción a Mutual Hazard Networks

Los modelos de progresión en cáncer (CPM) son métodos utilizados para predecir los futuros estados de un sistema y entender las restricciones en el orden de los eventos acumulados durante la progresión del tumor [1].

Entre esos modelos se encuentran los Mutual Hazard Networks (MHN), algoritmos de aprendizaje automático. Modelan la tasa de aparición de un suceso, como puede ser una mutación, mediante una tasa espontánea de adquisición y un evento multiplicativo que cada uno de estos sucesos puede tener sobre la tasa de otros sucesos mediante interacciones por pares que modelan dependencias promotoras e inhibidoras [2]. Eso representa la principal diferencia con los demás modelos de CPM; en MHN, los eventos no son determinísticos, si no que pueden afectar a la probabilidad de aparición de otros. Este modelo cuenta con una serie de supuestos: se sufre una acumulación de eventos irreversible, las muestras son independientes y los datos son homogéneos. Además, el momento en que se produce una observación (por ejemplo, un diagnóstico de cáncer) es independiente de los acontecimientos precedentes. En su lugar, se supone que los tiempos de observación siguen una distribución exponencial simple, que es un supuesto estadístico estándar que a menudo se hace por simplicidad.

No obstante, los MHN se han modificado recientemente para corregir el sesgo por observación [3]. En este nuevo enfoque, el acto de observar un acontecimiento se considera parte del proceso que se está modelando, lo que implica que el momento de una observación depende de los eventos que hayan ocurrido previamente. Por ejemplo, una acumulación más agresiva de eventos relacionados con el cáncer podría conducir a una detección más temprana. Es importante corregir por este sesgo porque la variable "observación del tumor" actúa como un "colisionador"; es decir, una variable influenciada por dos factores (en este caso, tanto las mutaciones genéticas como las circunstancias externas favorecen la observación del tumor). Ignorar este aspecto puede generar sesgos, como predecir que ciertas mutaciones suprimen otras cuando en realidad no lo hacen, además de ocultar relaciones causales reales. Para abordar este problema, se emplean fórmulas tensoriales avanzadas que permiten calcular la función de verosimilitud del modelo [4].

En este contexto, se distingue entre un modelo clásico (cMHN) y un modelo corregido para este sesgo (observation-aware MHN; oMHN). El modelo clásico asume que todos los tumores comienzan en un estado no mutado, que los eventos ocurren de forma irreversible y de uno en uno y que el tiempo de observación del tumor es aleatorio y desconocido. Este modelo se basa en cadenas de Markov en tiempo continuo, donde el tumor "salta" entre estados ($(0,0) \rightarrow (1,0)$, etc.) con ciertas probabilidades por unidad de tiempo, definidas mediante tasas de transición modeladas en una matriz Q . Esta matriz se construye utilizando los parámetros Θ , que representan las tasas base (Θ_{ii}) y las interacciones entre eventos ((Θ_{ij})). A partir de esta matriz, también se pueden calcular las probabilidades de los estados del tumor en el momento de observación, suponiendo una distribución exponencial fija del tiempo. Sin embargo, este modelo clásico no considera el impacto del sesgo por observación.

La matriz Θ captura la relación de interdependencia entre nodos (representando eventos), donde los riesgos de ciertos nodos afectan directamente a los riesgos de otros [2]. Así, la matriz describe las interdependencias y las intensidades de estas interacciones entre los eventos. Cada elemento Θ_{ij} representa el impacto directo del evento j sobre el evento i en términos del riesgo o probabilidad de un evento perjudicial. La diagonal representa los riesgos sin considerar interdependencias.

El nuevo modelo oMHN incorpora explícitamente el evento de observación en la red causal como un evento adicional. Este evento se introduce como un nodo adicional ($n + 1$) y su tasa base se fija en 1 para estandarizar la escala temporal. Además, se introduce un parámetro adicional que permite modelar cómo los eventos genéticos afectan la probabilidad de observar el tumor. En este modelo, una vez que ocurre el evento de observación, todos los eventos genéticos restantes "se congelan" (sus tasas se multiplican por 0), reflejando que no se observan más eventos genéticos después de la detección. A diferencia del modelo cMHN, el modelo oMHN divide los estados en dos subconjuntos: antes y después de la observación. La matriz Q se amplía para combinar los parámetros Θ y Ω , donde Ω captura cómo los eventos genéticos influyen en la tasa de observación. Sin embargo, la introducción de este nuevo parámetro genera un

problema de la no identificabilidad, que se aborda mediante una regularización simétrica controlada por el hiperparámetro λ [3].

Estos modelos trabajan con datos transversales, es decir, diferentes sujetos son tomados en tiempos diferentes (independientes). Estos datos se recogen en una matriz binaria en la cual las filas representan pacientes o sujetos y las columnas genes/SNPs/etc. Cada celda puede contener un 1 si el evento, alteración o mutación está presente o un 0 si no fue observado [5]. Algunos modelos pueden trabajar también con datos filogenéticos o datos longitudinales.

2. Comparación de implementaciones entre R y Python

2.1. Funcionalidad

2.1.1. Python: MHN

El módulo `mhn` de Python cuenta con las funciones originales de MHN. Cuenta con implementación en CUDA, la GPU de Nvidia, para poder acelerar el cálculo de la puntuación de la log-verosimilitud y su gradiente tanto para el espacio de estados completo como para el restringido.

El módulo de Python cuenta con los dos frameworks comentados anteriormente, el clásico y el corregido. Los autores recomiendan en la mayoría de los escenarios utilizar el modelo con el sesgo corregido (oMHN), ya que aumenta la interpretabilidad de los parámetros inferidos, aunque el modelado de los datos no se vea afectado. El primer paso es escoger la función de optimizado, siendo `cMHN0ptimizer()` el del modelo MHN clásico y `oMHN0ptimizer()` el del modelo corregido.

Se requiere que los datos de input estén en formato de matriz binaria, la cual se puede importar con el módulo `pandas` y la función `load_data_matrix()` o, en caso de que esté en un fichero csv, directamente con `load_data_from_csv()`.

Al tratarse de un algoritmo de aprendizaje automático, se requiere penalizar la inferencia de parámetros de modo que, en general, se prefiera un conjunto de parámetros disperso/parsimonioso. El resultado es explicar bien los datos de entrada de entrenamiento con el menor número posible de parámetros para que el modelo siga siendo generalizable a los datos de prueba. Así, los modelos generados son más sencillos e interpretables, evitando el sobreajuste. Para este paso, los autores han establecido dos formas de penalización:

- **Penalización estándar L1:** penaliza los parámetros individuales, fomentando que muchos de ellos sean exactamente cero. Esto conduce a un modelo disperso en el que sólo permanecen activos los parámetros más importantes.
- **Penalización simétrica personalizada:** penaliza los pares de parámetros que son simétricos en la matriz Θ (con respecto a la diagonal). El coste de que ambos parámetros de un par sean distintos de cero equivale a que sólo uno sea distinto de cero. Al fomentar la simetría en el modelo, se puede alinear mejor con las relaciones biológicas, como los efectos epistáticos.

En el artículo [3], los autores recomiendan elegir la penalización basándose en el conocimiento previo sobre la estructura de los efectos que se esperan en el tipo de datos con los que se está trabajando.

El siguiente paso es la elección de hiperparámetros de validación cruzada. Se prueba un rango de diferentes penalizaciones («lambdas») para identificar una que produzca un rendimiento óptimo en los datos retenidos. Se pueden definir los siguientes hiperparámetros: la intensidad mínima y máxima de la penalización («lambda») que se probará (usualmente se espera un lambda de $1/n$, siendo n el número de observaciones, por lo que se escogen un mínimo y máximo alrededor de ese valor), la cantidad de pasos (es decir, la longitud de la secuencia lambda) que hay que dar entre estos límites y la cantidad de folds en que se quieren dividir los datos para la validación cruzada. Se consigue alta precisión probando con un gran rango lambda con pequeños tamaños de paso y utilizando muchos folds, pero requiere más recursos informáticos y más tiempo de computación. Por el contrario, los resultados más rápidos utilizan

un rango más estrecho, menos pasos y menos folds, reduciendo el cálculo y sacrificando la precisión a la hora de encontrar la λ óptima. En el módulo, todo esto se puede especificar en la función `lambda_from_cv()`.

El último paso es entrenar el modelo final con el λ óptimo obtenido de la validación cruzada. Esto se consigue con `train()`, y los resultados se pueden guardar en formato csv y un fichero log en formato json con `result.save()`. El output principal se puede visualizar con `result.plot()`, mostrando la matriz Θ logarítmica de forma predeterminada.

2.1.2. Web app: evamtools

La web app de evamtools se encuentra en <https://www.iib.uam.es/evamtools/>, permitiendo estudiar acumulación de eventos en línea. Proporciona una interfaz gráfica al paquete de R, permitiendo al usuario una rápida construcción, manipulación y exploración de los modelos CPM.

La web comienza con el input del usuario, donde se puede introducir los datos subiendo un fichero o metiendo manualmente las frecuencias del genotipo. También se permite generar datos transversales de modelos CPM, diferenciando modelos que utilizan grafos acíclicos dirigidos para especificar restricciones de los MHNs, que modelan las relaciones inhibitoras/facilitadoras entre genes utilizando tasas de riesgo de referencia y efectos multiplicativos entre genes especificados en la matriz de riesgos mutuos log-*Theta*.

Si se sube un fichero de datos, la web muestra el conteo de genotipos y su respectivo histograma. Bajo opciones avanzadas se pueden seleccionar los distintos CPMs a utilizar y personalizar algunos de los parámetros. Los resultados se pueden visualizar en forma de los grafos acíclicos dirigidos o la matriz de MHN, además de la tabla de predicción de los modelos ajustados. Estos resultados se pueden descargar en formato RDS, tratándose de un objeto de R, al cual se puede importar.

2.1.3. R: evamtools

El paquete de evamtools de R permite calcular distintos CPMs de unos datos que se pueden generar (función `random_evam`) o importar desde un csv. Los CPMs que incluye son Conjunctive Bayesian Networks (CBM), Oncogenic Trees (OT), Disjunctive Bayesian Networks (DBN: OncoBN), Hidden Extended Suppes-Bayes Causal Networks (H-ESBCN) y MHN (Mutual Harzards Networks).

El paquete cuenta con 10 funciones exportadas:

- `evam`: Esta es la función principal para ajustar un modelo EvAM a un conjunto de datos proporcionados por el usuario (datos trasnversales). Genera un objeto que contiene la estructura del modelo ajustado, parámetros estimados y restricciones inferidas.
- `every_which_way_data`: Proporciona un conjunto de datos de prueba predefinido (22 datasets de datos de cáncer de los artículos [6] y [7]). Pueden ser usados como ejemplo o para validaciones.
- `ex_mixed_and_or_xor`: Es un conjunto de datos de ejemplo que incluye combinaciones de relaciones AND, OR y XOR entre eventos.
- `examples_csd`: Es un conjunto de datos de ejemplo diseñado para ilustrar modelos de datos seccionales cruzados (cross-sectional data). Estos pueden ser usados para probar las funcionalidades del paquete, sin la necesidad de preparar datos propios.
- `plot_evam`: Esta función genera representaciones gráficas de modelos ajustados EvAM y sus relaciones, visualizando transiciones entre genotipos en términos de probabilidades o tasas de transición. También permite personalizar el tipo de gráfica y los datos mostrados.
- `random_evam`: Esta función genera un modelo aleatorio de un tipo especificado, pudiendo personalizar parámetros como el número de genes, densidad del grafo, pesos, y distribución de errores. Devuelve un objeto con la misma estructura que la función `evam`. Esta función puede es util para simulaciones y pruebas.

- **runShiny:** Inicia la aplicación web Shiny integrada para explorar modelos de forma interactiva.
- **sample_evam:** Esta función genera muestras de genotipos a partir de un modelo EvAM ajustado (función `evam`) o aleatorio (función `random_evam`). Permite indicar el número de muestras a generar y el nivel de errores de genotipado.
- **evamtools-deprecated:** Sus funciones están en desuso dentro del paquete y serán reemplazadas en próximas actualizaciones. En ellas se encuentran:
 - **plot_CPMs:** Crea gráficos ajustados de EvAMs, utilizando los modelos ajustados y gráficos personalizados para las tasas y probabilidades de transición.
 - **sample_CPMs:** Obtiene muestras de los genotipos de los modelos de CPM. Opcionalmente, cuenta los genotipos de transición. Para todos los métodos excepto OT y OncoBN se puede solicitar `obs_genotype_transitions`, aunque se quitó de la versión web, por su poco uso y el aumento en el tiempo de procesado (por ello ya no se utiliza al tomar la muestra). Esta originalmente se obtiene contando las transiciones entre genotipos dos a dos en las simulaciones de Cadena de Markov de tiempo continuo. También se obtienen `state counts` al contar cuantas veces un genotipo fue visitado durante este proceso.
- **SHINY_DEFAULTS:** Define configuraciones predeterminadas para la aplicación Shiny del paquete. Es utilizada internamente por `runShiny`.

2.1.4. Comparativa entre las funcionalidades de `evamtools` y su página web

Tanto `EvamTools` como su página web permiten calcular un modelo cMHN a partir de un fichero que contenga los datos en forma de matriz binaria, y en el caso de la web también permite introducir los genotipos de los pacientes a mano. El cálculo del modelo se realiza con la función `evam` del paquete y nos permite especificar los parámetros: número de núcleos y λ ; esta última también la podemos especificar en la página web. El output de ambos es similar: se obtiene la matriz $\log-\Theta$, los ratios de transición, las probabilidades de transición,... así como las frecuencias relativas de los genotipos predichos. Para poder seguir trabajando con el output que devuelve Shiny nos debemos descargar el objeto e importarlo en R, de manera que es mucho más cómodo correrlo directamente en R. Además, hay veces que los datos de las figuras se solapan unos con otros impidiendo la correcta visualización y comprensión de los resultados por lo que es casi una obligación su importación a R.

2.1.5. Comparativa entre las funcionalidades de `evamtools` y `mhn`

Respecto al paquete `mhn` de Python y `evamtools` de R, mientras que este último sólo permite calcular el modelo cMHN, en Python se puede calcular tanto el cMHN como el oMHN aplicando la corrección del sesgo. A la hora de calcular el modelo, `mhn` en Python contiene la función `lambda_from_cv()` que nos permite calcular la λ óptima para nuestro conjunto de datos indicando las λ s máxima y mínima, el número de pasos y el número de folds. Sin embargo, hemos observado que para datasets pequeños hay veces que el cálculo de la λ da error (Zero Division Error). En el caso de `evamtools`, no hay ninguna función en el paquete que nos permita realizar este cálculo. Tras el cálculo del modelo, Python sólo devuelve la matriz $\log-\Theta$, mientras que R además devuelve la matriz $\log-\Theta$ exponencial, los ratios de transición, las probabilidades de transición,... Una vez calculado el modelo, a partir de él se pueden generar datos sintéticos empleando la función `sample_artificial_data()` de Python al igual que se puede hacer con `sample_evam()` de R. Al calcular el modelo en R con `evam` podemos especificar el parámetro `paths_max=TRUE`, el cual nos devuelve las probabilidades que tienen cada uno de esos paths. En Python, podemos calcular la probabilidad de que una mutación haya precedido a otra sabiendo que ambas se encuentran en el presente mutadas mediante la función `sample_trajectories()`; y también nos permite predecir el siguiente evento teniendo en cuenta un genotipo particular con la función `compute_next_events_probs()`. Estas dos últimas funcionalidades no las presenta R.

2.2. Velocidad

Dentro del paquete `evamtools` en R, los datos pueden manejarse tanto como data frames como matrices, funcionando de igual manera y sin diferencias significativas en el tiempo de ejecución. La función `evam` permite especificar el número de núcleos a utilizar para la computación del modelo. Tras medir el tiempo de ejecución del modelo con tres conjuntos de datos de distintos tamaños (81, 500 y 3662 observaciones con 6, 12 y 12 genes respectivamente), se observa una ligera tendencia: a mayor número de núcleos, el tiempo de computación disminuye. No obstante, al alcanzar una cierta cantidad de núcleos, que varía según el ordenador, el tiempo de ejecución deja de reducirse.

El módulo de Python `mhn` cuenta con una implementación que utiliza CUDA para aumentar la velocidad de cálculo. Sin embargo, debido a las características de nuestros ordenadores, no hemos podido comprobar esa reducción en el tiempo de computación, trabajando solo con la versión estándar. En esta versión, dos pasos consumen la mayor parte del tiempo de computación: la búsqueda del λ óptimo y el entrenamiento del modelo. La búsqueda del λ óptimo es el paso más lento; en el modelo `cMHN`, este proceso es varios segundos más rápido que en `oMHN`. Esta diferencia también se aprecia en el entrenamiento del modelo, aunque es menor, variando entre décimas de segundo y un segundo.

En cuanto a la aplicación web, es la herramienta más lenta de las tres examinadas. Un conjunto de datos con aproximadamente 80 observaciones y 6 genes tarda entre 5 y 15 segundos en calcular el modelo `MHN`, utilizando los parámetros predeterminados. Con conjuntos de datos de 500 observaciones, ese tiempo supera los 3 minutos, y en ocasiones puede tardar más de media hora en mostrar en pantalla la matriz de resultados.

2.3. Ventajas y desventajas

En cuanto al módulo de Python, una gran desventaja ha sido el uso de CUDA. Pese a que no sea necesario para poder utilizar las funcionalidades, sí aumenta la velocidad de cálculo, lo que puede suponer una gran ventaja al trabajar con datasets grandes. No obstante, el módulo solo funciona con CUDA en una versión de Python 3.8.10, limitando así su uso. También pueden dar distintos errores en su instalación, y pese a que los autores tengan un apartado explicando la forma de instalar CUDA para `mhn`, no termina de solucionar todos los errores que salen y se requiere de mucho tiempo, conocimiento y paciencia para lograrlo. Adicionalmente, el módulo de Python parece tener problemas con ficheros de datos relativamente pequeños debido al cálculo del λ óptimo que realiza, como ocurre con el documento de `BRCA_ba_s.csv`.

Respecto a la web app, es intuitiva de utilizar gracias a su tutorial y simple en cuanto a sus opciones, pero no es la herramienta apropiada para análisis más exhaustivos y sistemáticos más allá de pequeños experimentos computacionales. El tiempo de ejecución es relativamente alto, por lo que es preferible realizar ese tipo de análisis con un script de R mediante el uso del paquete de `evamtools`. Se muestra un aviso cuando el fichero de entrada cuenta con más de 7 genes, limitación que no está presente en el paquete de R.

Finalmente, el paquete de R es la herramienta más completa de las analizadas en este trabajo al contar con distintos modelos de acumulación de eventos y ser utilizable para análisis sistemáticos. Sin embargo, no cuenta con la última versión de `MHN` corregida por el sesgo del colisionador.

3. Exploración de posibles integraciones

3.1. Integración del código en Python en R (Evamtools)

En muchas ocasiones, es necesario combinar herramientas y librerías de diferentes lenguajes de programación con el objetivo de aprovechar sus fortalezas específicas. La integración entre los lenguajes de Python y R puede ser clave, permitiendo así disponer de las funcionalidades de ambos paquetes (`mhn` y `evamtools`) para poder desarrollar proyectos más complejos.

Con este propósito, el paquete `reticulate` de R proporciona una interfaz versátil que permite ejecutar código de Python directamente desde el entorno de R. `reticulate` facilita la interoperabilidad entre ambos lenguajes, permitiendo acceder a librerías, funciones y objetos de Python como si fueran nativos en R. Esta capacidad resulta especialmente útil cuando se desean utilizar paquetes especializados de Python, como `mhn`, dentro de un flujo de trabajo basado en R. `reticulate` es la opción elegida debido a su simplicidad, flexibilidad y su fácil acceso a las numerosas librerías de Python.

En este proyecto, hemos explorado el uso de `reticulate` para llamar al código de Python relacionado con el paquete `mhn` desde R. Los pasos seguidos fueron:

1. Configuración del entorno: Asegurar que Python esté instalado y accesible desde R mediante la configuración del intérprete Python en `reticulate` en la imagen de Docker.
2. Instalación del paquete `mhn` en Python
3. Ejecución de código de Python desde R: Llamar a las funciones del paquete `mhn`
4. Transferencia de datos entre lenguajes: Probar la transferencia de datos y verificar la correcta ejecución de las funciones.

En Python, se nombra `cMHN` como el modelo clásico de MHN, el cual es el implementado en `evamtools`. Por ello, hemos querido comparar los resultados de ambas formas, utilizando el mismo dataset en Python y en R. Los resultados han mostrado la misma tendencia en ambas opciones (relación positiva o negativa entre los genes), aunque los valores concretos varían. Al comparar los dos códigos, la diferencia radica en el cálculo de un λ óptimo ajustado a los datos en el caso de Python, mientras que `evamtools` permite especificar un λ y, en caso contrario, utiliza el ajuste genérico de $1/n$. Al añadir manualmente el valor de λ calculado por `mhn` en `evam`, los resultados de la matriz en ambos paquetes sí son idénticos.

```
library("reticulate")

# Configurar reticulate para usar Python 3 y verificar su configuración:
use_python("/usr/bin/python3", required = TRUE)
py_config()

###

# Importar librerías a usar numpy y matplotlib
mhn <- import("mhn")
np <- import("numpy")
plt <- import("matplotlib.pyplot")
random <- import("random")

# Usar reticulate para obtener el atributo __version__ del paquete mhn e imprimirla
mhn_version <- py_get_attr(mhn, "__version__")
print(mhn_version)

###

# Cargar el archivo CSV en un dataframe con 500 observaciones, en R
data <- read.csv('LUAD_500.csv')

# Correr el código de python con la función "py_run_string"
py_run_string("
import random
import pandas as pd
```

```

input = pd.read_csv('LUAD_500.csv')
print (input.head())

print('Number of observations:', len(input))
print('Number of events:', len(input.columns))

print('Event frequencies:')
input.sum(axis=0) / len(input)

print('istribution of active event counts across observations:')
input.sum(axis=1).value_counts().sort_index()

# En este caso no es necesario coger una muestra de 500 observaciones, por lo que:
input_subset = input

# Generar resultados
result = input_subset.sum(axis=1).value_counts().sort_index()
print (result)
")

# Convertir el dataframe de R a un dataframe de pandas en Python
py$data <- r_to_py(data)

# Verificar cómo está estructurado el DataFrame en Python
py_run_string("
print(result)
")

# Acceder a los resultados de Python en R
input_subset_python <- py$input_subset

# Convertir la Serie de pandas a una lista en Python
py_run_string("
result_list = result.tolist() # Convertir la Serie a lista
")
# Acceder a la lista desde R
result_list <- py$result_list
print(result_list)

# Ejecutar las funciones de Python usando reticulate
py_run_string("from mhn.optimizers import cMHNOptimizer")
py_run_string("cMHN_opt = cMHNOptimizer()")

# Cargar los datos en los optimizadores
py_run_string("cMHN_opt.load_data_matrix(input_subset)")

# Establecer las penalizaciones para cMHN
py_run_string("cMHN_opt.set_penalty(cMHN_opt.Penalty.L1)")

# Verificar si se cargaron correctamente los datos y la configuración
py_run_string("print(cMHN_opt)")

###

```



```
py_run_string("
import random
import pandas as pd
from mhn.optimizers import cMHNOptimizer

# Convertir el dataframe a un objeto manejable en Python
input = data

# Estadísticas básicas del conjunto de datos
print('Number of observations:', len(input)) # Número de individuos
print('Number of events:', len(input.columns)) # Número de eventos

print('Event frequencies:')
print(input.sum(axis=0) / len(input)) # Frecuencias de eventos

print('Distribution of active event counts across observations:')
print(input.sum(axis=1).value_counts().sort_index()) # Distribución

result = input_subset.sum(axis=1).value_counts().sort_index()

# Inicializar el optimizador
cMHN_opt = cMHNOptimizer()

# Cargar los datos en el optimizador
cMHN_opt.load_data_matrix(input)

# Establecer las penalizaciones para cMHN
cMHN_opt.set_penalty(cMHNOptimizer.Penalty.L1)

# Verificar la configuración del optimizador
print(cMHN_opt)
")

# Para acceder a 'result' desde R
result <- py$result
print(result)

#####

# Calcular el lambda óptimo que usa python
py_run_string('
lambda_min= 0.1 / len(input_subset)
lambda_max = 100 / len(input_subset)

print("Minimum lambda:", lambda_min)
print("Maximum lambda:", lambda_max)

n_cv_steps = 5

import numpy as np

lambda_sequence = np.exp(np.linspace(
    np.log(lambda_min + 1e-10), np.log(lambda_max + 1e-10), n_cv_steps))
```

```
print("Lambda sequence:")
lambda_sequence

n_cv_folds = 3

cMHN_opt.set_device(cMHNOptimizer.Device.CPU)

cMHN_lambda = cMHN_opt.lambda_from_cv(
    lambda_min=lambda_min, lambda_max=lambda_max, steps=n_cv_steps, nfolds=n_cv_folds,
    show_progressbar=True)

import matplotlib.pyplot as plt

# plot original lambda sequence
plt.scatter(lambda_sequence, np.full(len(lambda_sequence), 1), color="black")

# plot cross-validated lambdas
plt.scatter(cMHN_lambda, 2, color="red")

# labels
plt.text(cMHN_lambda, 1.9, "cMHN", ha="center")

# log scale
plt.xscale("log")
plt.yticks([])
plt.ylabel("")
plt.show()
')
```

Obtener las secuencias de lambda desde Python a R

```
lambda_sequence <- py$lambda_sequence
cMHN_lambda <- py$cMHN_lambda
```

#####

Obtener el resultado de cMHN utilizando la lambda previa

```
py_run_string("
# train cMHN
cMHN_opt.train(lam=cMHN_lambda)
# save output
cMHN_opt.result.save(filename='cMHN.csv')

import json
import pprint

with open('cMHN_meta.json') as f:
    cMHN_log = json.load(f)

pprint.pprint(cMHN_log)
")
```

#####

Obtener el gráfico cMHN

```

py_run_string("import matplotlib.pyplot as plt")

py_run_string("cMHN_opt.result.plot()")

# Mostrar el gráfico generado
py_run_string("plt.show()")

# O mostrar la matriz generada
py_run_string("print(cMHN_opt.result)")

###
#####
#####
## Ahora los resultados obtenidos con la función evam

library("evamtools")

datos <- evam(data, methods = "MHN")
datos$MHN_theta      # Se corresponde con cMHN_opt.result en Python
# No da exactamente los mismos resultados
# CUIDADO. El orden de las filas y las columnas no coincide en Python y R.

# Sin embargo, al repetir evam utilizando la lambda de Python...
datos2 <- evam(data, methods = "MHN", mhn_opts = list(lambda = cMHN_lambda))
datos2$MHN_theta
# ... sí que coinciden los resultados.

```

3.2. Modificar evamtools para incluir la corrección del sesgo

Dentro de evamtools, el modelo cMHN se encuentra codificado en el fichero `MHN__ModelConstruction.R`, estando relacionado con la construcción y manipulación de la matriz de transición (Q) basada en los parámetros de Θ .

Para poder incluir oMHN en evamtools, primero se debe agregar el evento de observación en la matriz Θ . Para ello, se debe modificar la función `Random.Theta` para agregar una fila y una columna adicionales que representen el evento de observación y ajustar los valores iniciales para que el evento de observación tenga parámetros razonables en la matriz.

```

Random.Theta <- function(n, sparsity=0){
  Theta <- matrix(0,nrow=n+1,ncol=n+1) # Agregar una dimensión adicional

  diag(Theta) <- NA
  nonZeros <- sample(which(!is.na(Theta)), size=((n+1)^2 - (n+1))*(1 - sparsity))

  Theta[nonZeros] <- rnorm(length(nonZeros))
  diag(Theta) <- rnorm(n+1) # Ajustar la diagonal

  return(round(Theta,2))
}

```

A continuación se deben incluir las interacciones entre eventos y observación, modificando `Q.Subdiag` para tener en cuenta la nueva fila y columna en Θ .

```

Q.Subdiag <- function(Theta, i) {
  row <- Theta[i,]
  n <- length(row) - 1 # Excluye el evento de observación
  s <- exp(row[i]) # Base rate Theta_ii

  # Caso especial: si i es el evento de observación
  if (i == n + 1) {
    # Define cómo el evento de observación afecta a los demás
    s_obs <- exp(row[1:n]) # Interacciones con los eventos principales
    return(c(s, s_obs))
  }

  # Caso general: eventos no relacionados con la observación
  for (j in 1:(n + 1)) {
    if (i != j) {
      # Multiplica la tasa base por la interacción Theta_ij
      s <- c(s, s * exp(row[j]))
    }
  }

  return(s)
}

```

Lo siguiente es actualizar la construcción de Q. Para ello, hay que incorporar a la función Build.Q las interacciones del evento de observación con el resto de los eventos.

```

Build.Q <- function(Theta) {
  n <- nrow(Theta) - 1 # Número de eventos sin incluir el de observación
  Subdiags <- list()

  # Construir las subdiagonales incluyendo el evento de observación
  for (i in 1:(n + 1)) {
    subdiag <- Q.Subdiag(Theta, i)
    expected_length <- 2^(n + 1 - i) # Tamaño esperado de la subdiagonal
    Subdiags[[i]] <- ifelse(length(subdiag) >= expected_length,
                             subdiag[1:expected_length],
                             c(subdiag, rep(0, expected_length - length(subdiag))))
  }

  # Crear la matriz de transición esparsa
  Q <- Matrix::bandSparse(
    2^(n + 1), # Tamaño de la matriz (incluye observación)
    k = -2^(0:n), # Posiciones de las subdiagonales
    diagonals = Subdiags # Subdiagonales calculadas
  )

  # Calcular la diagonal principal (tasas de salida)
  diag(Q) <- 0 # Inicializa la diagonal principal en 0
  for (i in 1:(2^(n + 1))) {
    diag(Q)[i] <- -sum(Q[i, -i], na.rm = TRUE) # Suma negativa de las tasas de salida
  }

  return(Q)
}

```

```
}
```

Bibliografía

- [1] Ramon Diaz-Uriarte y Iain G. Johnston. *A picture guide to cancer progression and monotonic accumulation models: evolutionary assumptions, plausible interpretations, and alternative uses*. 2023. DOI: [10.48550/ARXIV.2312.06824](https://doi.org/10.48550/ARXIV.2312.06824).
- [2] Rudolf Schill et al. "Modelling cancer progression using Mutual Hazard Networks". En: *Bioinformatics* 36.1 (jun. de 2019), págs. 241-249. ISSN: 1367-4803. DOI: [10.1093/bioinformatics/btz513](https://doi.org/10.1093/bioinformatics/btz513). eprint: https://academic.oup.com/bioinformatics/article-pdf/36/1/241/48981322/bioinformatics_36_1_241.pdf.
- [3] Rudolf Schill et al. "Correcting for Observation Bias in Cancer Progression Modeling". En: *Journal of Computational Biology* 31.10 (oct. de 2024), 927-945. ISSN: 1557-8666. DOI: [10.1089/cmb.2024.0666](https://doi.org/10.1089/cmb.2024.0666).
- [4] Rudolf Schill et al. "Overcoming Observation Bias for Cancer Progression Modeling". En: *Research in Computational Molecular Biology*. Ed. por Jian Ma. Cham: Springer Nature Switzerland, 2024, págs. 217-234. ISBN: 978-1-0716-3989-4. DOI: <https://doi.org/10.1007/978-1-0716-3989-414>.
- [5] Ramon Diaz-Uriarte. *EvAM-Tools: methods' details and FAQ*. 2023.
- [6] Ramon Diaz-Uriarte y Claudia Vasallo. "Every which way? On predicting tumor evolution using cancer progression models". En: *PLOS Computational Biology* 15.8 (ago. de 2019). Ed. por Arne Traulsen, e1007246. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1007246](https://doi.org/10.1371/journal.pcbi.1007246).
- [7] Juan Diaz-Colunga y Ramon Diaz-Uriarte. "Conditional prediction of consecutive tumor evolution using cancer progression models: What genotype comes next?" En: *PLOS Computational Biology* 17.12 (dic. de 2021). Ed. por Rainer Spang, e1009055. ISSN: 1553-7358. DOI: [10.1371/journal.pcbi.1009055](https://doi.org/10.1371/journal.pcbi.1009055).